

LMS Platform: Architecture and Plan

Status: draft v1 (2026-10-01). Source of truth for the two developers and for Claude Code.

1. Context and goals

We currently run a vendor-hosted LMS: high cost, no course customization, no control over auth or learner analytics. We are building a native LMS that is end to end: auth and RBAC, a hosted web player that reads course content as data (no SCORM/xAPI packaging), AI-generated and human-reviewed content in a 9-stage structure, our own event analytics, spaced review, a tutor chatbot, a support bot and a voice tutor.

2. Decisions so far

Area	Decision
Customers	Both direct individual learners (B2C) and company learners (B2B)
Versioning	Learner progress migrates by stable step ID across course versions
Access after contract end	Open. Entitlement field <code>access_after_expiry</code> (<code>none/read_only/full</code>); default <code>read_only</code>
Analytics audience	Internal team only (V1); events carry <code>org_id</code> so client dashboards stay possible
Vendor cutover	Handled by the company, out of scope here
Team	2 developers + Claude Code; separate content team; non-technical reviewers
Timeline	~3 months to V1
Frontend	Next.js (App Router), React, TypeScript, HTML, CSS
Hosting	GCP (Cloud Run + Cloud SQL). Azure equivalents: Container Apps + Azure Database for PostgreSQL
Export	No SCORM/xAPI export
Languages	Multilingual (language list open)
Instructors	Monitor and support only, assigned to specific learners, no content edits
Tutor memory	Current session only
Voice	Voice version of the tutor, built as a thin layer over the text tutor

3. System architecture

```
Browser (Next.js, React, TypeScript, next-intl)
|
Next.js app on Cloud Run (one container)
|- Auth: managed IdP (orgs, roles, SAML/OIDC later)
|- Player API: content, progress, attempts, grading
|- Admin consoles: super admin, org admin, instructor, content, review
|- Tutor / support bot API (retrieval + Claude API)
|- Voice gateway (streaming STT -> tutor -> TTS)
|- Event capture
|
|- Cloud SQL Postgres (relational + jsonb content, pgvector for retrieval)
|- Cloud Tasks / Scheduler: spaced review, notifications, version migrations
|- Email provider: reminders (with consent)
|- Cloud Storage + CDN: media, transcripts
|- Claude API: authoring skills, tutor, support bot
```

A full layered diagram (users, frontend, auth and RBAC, backend, data, content pipeline, external services) is in [architecture-diagram.png](#) and on the last page of [ARCHITECTURE.pdf](#).

Principles: one codebase, buy rather than build (auth, email, speech), schema package as single source of truth, every authorization decision server-side.

4. Roles and RBAC

Roles are assigned at a scope (platform, organization, cohort/learner set), never globally.

Role	Scope	Capabilities	Landing page
Super admin	Platform	Orgs, users, entitlements, roles, internal analytics, settings	Platform console
Org admin	Organization	Seats, learners, course assignment, org progress	Org dashboard
Instructor	Assigned learners	View progress and stuck points of assigned learners, message them. No content edits	Assigned-learner view
Content creator	Platform	Run generation skills, track module status, respond to review comments	Content workspace
Reviewer	Platform	Approve/reject modules (non-technical review page)	Review queue
Learner	Self	Courses, tutor, own history	Continue learning, due reviews, assignments

Implementation rules:

- `role_assignments(user_id, role, scope_type, scope_id)`; one user may hold several roles.
- A single `can(user, action, resource)` function is called by every route handler and server action.
- `/` redirects by role priority; multi-role users get a role switcher.
- Instructor access is limited to rows in `instructor_assignments(instructor_id, learner_id)`.
- Test rule: every endpoint has a test proving a user from another org (or an unassigned instructor) gets 403.

5. Data model

Identity and access: `Organization`, `User`, `Membership`, `RoleAssignment`, `InstructorAssignment`, `Entitlement(owner=user|org, course, valid_from, valid_to, seats, access_after_expiry)`.

Content: `Course` -> `CourseVersion` (immutable once published) -> `Module` -> `StageBlock`. Every block has an immutable `step_id`. Localized text lives in `ContentLocale(step_id, version, locale, payload)`, so step IDs are language-agnostic.

Learning: `Enrollment`, `StepProgress(enrollment, step_id, status)`, `Attempt + ItemResult`, `ReviewItem` (spaced review schedule), `TutorSession` (current session only, retention-limited).

Analytics: `Event` (append-only, always with `org_id` nullable for individuals, `user_id`, `step_id`, `locale`, type, payload).

Version migration by step ID

On publish, a job maps progress from the old version through an old-to-new step-ID map:

- Removed steps are archived, never deleted from history.
- New steps appear as pending.
- A materially changed mastery-gate item bank resets that gate; otherwise it keeps its pass.
- The publish step reports how many learners each change affects before it goes live.

6. Content schema (Zod, packages/content - schema)

Typed block per stage with required fields:

- Hook: scenario, media, objective link
- Explainer: limited concepts, visual, transcript
- Worked example: steps with fading levels (full -> partial -> independent)
- Guided practice: items, tiered hints, feedback rules, auto-grading
- Scenario lab: environment type, starting state, rubric (later)
- Mini project: brief, deliverable spec, rubric (later)

- Mastery gate: item bank, pass threshold, retry policy, remediation route
- Reflection: prompts
- Spaced review: concept tags, schedule, transfer variants

Module metadata: objectives (Bloom level, knowledge type), prerequisites, source references, reviewer, version, review-by date, stages included, skill versions used. V1 ships stages 1-4, 7, 8, 9; every module states which stages it has.

7. Content pipeline (Claude Code skills)

Sources -> Ingest -> Brief (approved) -> Generate per stage -> Auto-validate
-> Repair (max 3) -> Human review -> Publish -> Monitor

Repo layout:

```
.claude/skills/
  module-generator/      orchestrator
    stage-<name>/        one skill per stage (shared craft rules + examples)
    course-<domain>/      per-course terminology, pitfalls, source rules
packages/content-schema/ Zod schemas
packages/validators/     validate-module CLI
content/<course>/<module>/ generated JSON, source refs, review record
```

Validators (cheapest first): schema; claim-level grounding to source chunks; objective -> activity -> assessment alignment; pedagogy rules (concept count, fading levels, tiered hints, distractor rationales, durations); code execution; independent solve of every quiz item by a separate call with no key; duplicate/leakage check between practice and gate.

Human review (non-technical page) sees only what machines cannot verify: factual accuracy and authenticity against the cited source, realism of scenarios. Comments are tagged **factual | pedagogy | domain | tone | structure** with severity and block ID.

Improvement loop

Reviewer finding	Fix lives in
Wrong/outdated fact	Source library, then regenerate
Weak pedagogy	Shared stage skill
Domain-specific error	Course skill
Machine-checkable problem	New validator

Weekly triage turns repeated comments into skill/validator changes. Each fixed mistake becomes a regression case; 3-5 benchmark modules are regenerated after every skill change. Track first-pass approval rate, comments per module and validator failures per module. Skills are in git, so a change that doesn't help is reverted.

8. Tutor, support bot and voice

Course tutor (in player). Retrieval over published module content and its source chunks, restricted to the learner's entitled courses. Cites what it used; says so when nothing relevant is found; never reveals or solves mastery-gate items; knows the learner's current stage. Memory is **current session only**.

Support bot (elsewhere). Separate small knowledge base (account, access, how-to); no course content; escalates to a human.

Shared. Per-user rate limits and cost caps; disclose it is an AI; retrieved text treated as data, never instructions (prompt-injection defense); session logs follow a retention rule.

Voice tutor. Text-first tutor, then streaming STT -> same tutor backend -> TTS. Managed speech provider (GCP Speech-to-Text/Text-to-Speech, pending language coverage check). No audio stored by default, transcripts only, explicit microphone consent. Stretch goal inside the 3 months.

9. Multilingual

Decision needed early because it shapes schema, pipeline and staffing.

- **Step IDs and structure are language-agnostic.** Progress follows `step_id`, so a learner can switch language mid-course.
- **Translate in the authoring pipeline, not at runtime.** Author in a base language, then produce each locale as a reviewed translation (translate, do not regenerate, so item banks stay equivalent). Run the same validators per locale, including the independent solve. Runtime machine translation of graded content is not allowed.
- **Reviewers per language.** Each published locale needs a reviewer fluent in it. This is a staffing constraint, not a code one.
- **UI i18n** with next-intl; locale in user profile, with URL-level locale routing.
- **Tutor:** multilingual embeddings; answers in the learner's language while citing the source passage.
- **Voice:** supported languages are limited by the STT/TTS provider; check before promising a language.
- **Code samples and tool names stay in the source language.**
- Content with no approved translation for a learner's locale falls back to the base language with a visible notice.

10. Analytics (internal)

Event capture from the player: stage entered/completed, attempts, hint use, gate pass/fail, tutor use, drop-off. First views: completion, first-attempt gate pass rate, drop-off by stage, item pass rates (to catch bad items or bad keys). Metrics are defined in writing before being shown to anyone outside the team.

11. Spaced review

`ReviewItem` rows scheduled by a job (Cloud Tasks/Scheduler); in-app due list on the learner landing page; email reminders only with consent and an unsubscribe path. Transfer variants come from the module's spaced-review block.

12. Security and privacy

- Server-side authorization everywhere; cross-tenant tests are mandatory.
- Managed IdP; SAML/OIDC SSO kept possible for company clients.
- Data processing terms, retention rules and deletion-on-request need legal input before the first paying company client; applicable law depends on where learners live (to verify).
- Secrets in Secret Manager; audit log for admin actions.

13. Working with Claude Code

- `CLAUDE.md` holds conventions and the invariants: stable step IDs, immutable versions, `org_id` on events, schema package is truth.
- Tests first for migration by step ID, entitlements/access, grading and mastery-gate logic, and RBAC.
- Small PRs per area, each reviewed by a human; CI runs tests and content validators.

14. 3-month plan (2 developers)

Weeks	Deliverable
1-2	Repo, CI, Cloud Run + Cloud SQL, Zod schemas for 9 stages + metadata, Postgres schema, step-ID/version rules, i18n scaffolding
3-4	Auth, orgs, RBAC, entitlements, role-based landing pages, super admin basics
5-7	Player for stages 1-4, 7, 8; progress and attempts
8-9	Event capture, internal analytics, instructor view
10-11	Review page for non-technical reviewers, validators, skill scaffolding, first modules; tutor (text)
12	Spaced review + reminders, support bot, hardening, pilot. Voice only if time allows

Not in V1: scenario labs, mini projects, client-facing dashboards, company SSO, fully generated personalized content, vendor course migration tooling.

15. Risks

- Scope versus timeline: cut order is voice, then reminders, then support bot, then review page polish.
- The 9-stage pattern is unproven; run an A/B test on one topic against a lighter structure and compare completion, first-attempt gate pass rate and a 1-2 week retention check. Needs the vendor completion baseline.
- Multilingual multiplies review work; each language needs reviewers.
- Reviewer capacity limits how much content ships.

16. Open questions

- 1 Access after contract end (default **read_only** until decided).
- 2 Which languages, and the base authoring language?
- 3 Are org admin, content creator and reviewer confirmed for V1? (assumed yes)
- 4 Vendor completion rates for the A/B baseline.
- 5 Auth provider (Clerk, Auth0, WorkOS) and ORM (Prisma or Drizzle).
- 6 Whether reviewers exist per language.

LMS Platform: System Architecture

Draft v1 (2026-10-01). Requests flow top to bottom; every request passes authentication, RBAC and entitlement checks.

1. Users

Roles are scoped (platform / org / assigned learners)

2. Frontend

Next.js, React, TS
next-intl (multilingual)
Landing page by role

3. Auth + RBAC

Server-side on every request

4. Backend

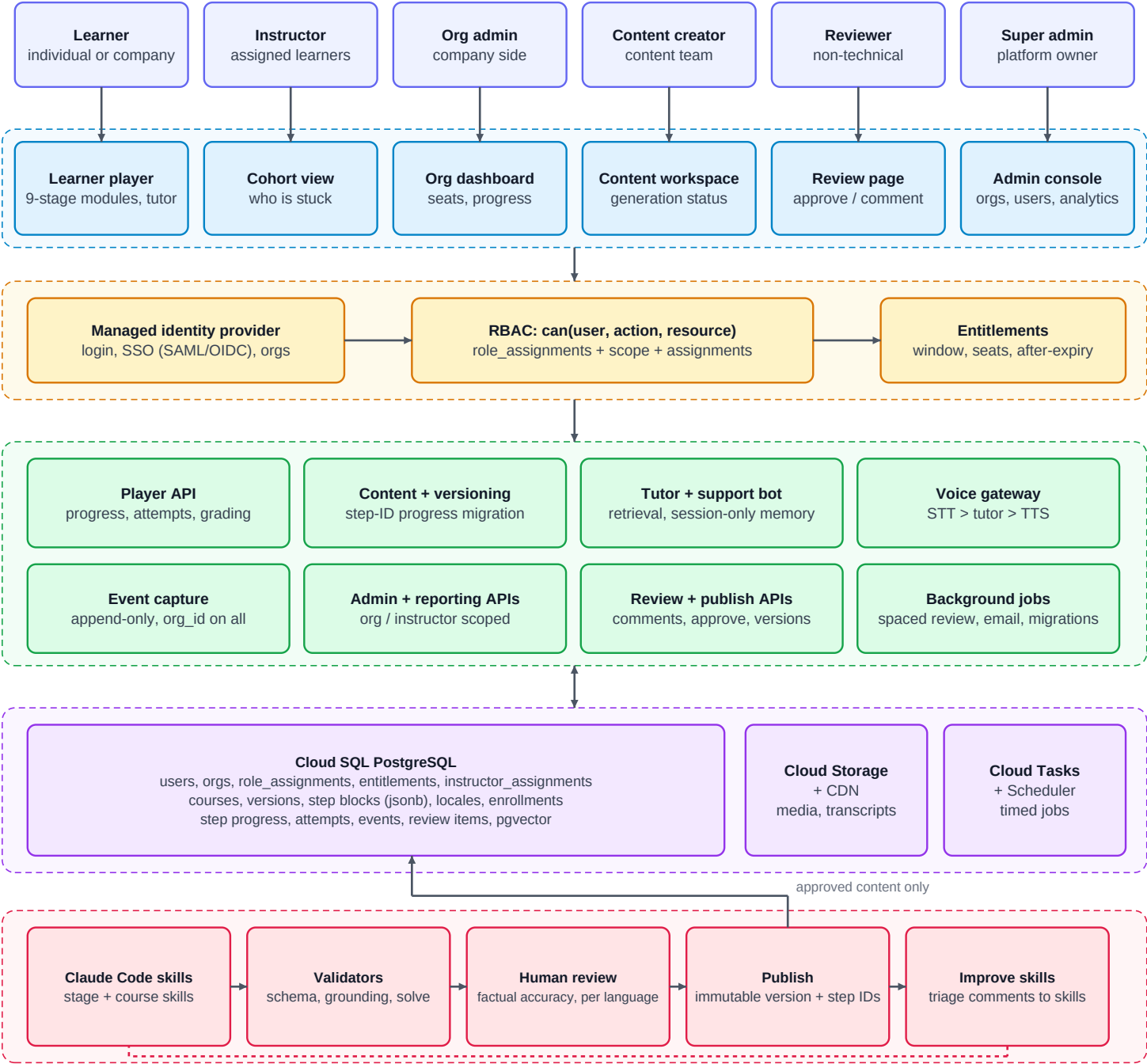
Next.js route handlers and server actions on GCP Cloud Run (one codebase)

5. Data

Managed GCP services

6. Content

Claude Code skills generate; people review and improve



Rules the design depends on

Every call: authenticate, then can(), then entitlement.

Step IDs never change across versions; progress migrates by step ID.

org_id is on every event.

Graded content is reviewed and published, never generated or translated at runtime.

Tutor never reveals mastery-gate answers.

External services

- Claude API**
authoring, tutor, support bot
- Speech provider**
STT / TTS, per language
- Email provider**
reminders, with consent

Claude Code skills also call the Claude API.